

Index

[1. Introduction - Webhooks Subscription](#)

[1.1 Current Version \(V3\)](#)

[2. New Version \(V4\)](#)

[2.1 Subscribing](#)

[2.2 Listing Subscriptions](#)

[2.3 Listing a specific subscription](#)

[2.4 Updating the callbackURL of a subscription](#)

[2.5 Removing a subscription](#)

[2.6 Additional information to document](#)

[2.7 Webhook Events Signature](#)

[2.7.1 Public Key Endpoint](#)

[3. IP Whitelisting](#)

[4. Webhook retry events](#)

[General information](#)

[Preview amount of events to be retried](#)

[Execute retry of events](#)

1. Introduction - Webhooks Subscription

Webhooks subscription endpoint is through where clients register a callback url to receive events of their transactions.

1.1 Current Version (V3)

The documentation for the current version being used by our clients (v3) is available in

- <https://docs.bitso.com/mm-xb/docs/webhooks>
- <https://docs.bitso.com/bitso-payouts-funding/docs/webhook-notifications-pix>

The endpoint specified in the documentation is POST <https://api.bitso.com/api/v3/webhooks/>

2. New Version (V4)

The V4 API version adds improvements on the previous version; it behaves in a way where it is now possible to execute operations that previously weren't as well as makes better use of the HTTP verbs to make request intentions clearer.

List of changes:

- /api/ removed from endpoint in accordance with RFC-39, the domain (api.bitso.com) already contains api, no need to duplicate it
- Adding a new subscription through http POST method
- Add individual callback url for each eventType is supported
- Deleting a subscription through http DELETE method
- List all user subscriptions through http GET method
- List individual user subscription through http GET method
- HTTP Responses

2.1 Subscribing

Now it is possible to subscribe to individual event types, possible values are:

- funding
- withdrawal
- dynamic_qr_code

Users that subscribe through the api version 4 will automatically start receiving payload signatures in the header of requests.

POST Request to /v4/webhooks/

Request Payload Example:

```
JavaScript
{
  "callback_url":
  "https://webhook-test.com/026b93e74068b30304e45a2488b6f486",
  "events": ["funding", "withdrawal"]
}
```

A 201 created status code should be returned, as well as the list of webhooks created and a message indicating the success of the operation

Response

Status Code: 201 OK

Body:

```
JavaScript
{
  "message": "Webhook successfully created",
  "webhooks": [
    {
      "id": 275,
      "event": "funding",
      "callback_url":
      "https://webhook.site/4fe0b84f-0674-4a62-8f7f-6b714037a5d1"
    }
  ]
}
```

}

2.2 Listing Subscriptions

To list all subscriptions for your user execute a GET request to /v4/webhooks/

If subscriptions exist for the user, the response looks as follows:

Response

Status Code: 200 OK

Body:

```
Java
[
  {
    "id": 11,
    "event": "funding",
    "callback_url":
    "https://webhook-test.com/026b93e74068b30304e45a2488b6f486"
  },
  {
    "id": 12,
    "event": "withdrawal",
    "callback_url":
    "https://webhook-test.com/026b93e74068b30304e45a2488b6f486"
  }
]
```

If the user doesn't have any subscriptions a 204, no content status code will be returned.

2.3 Listing a specific subscription

It is also possible to retrieve information on just a specific subscription by executing a GET request to `/v4/webhooks/{subscriptionId}`

If subscriptions exist for the user, the response should be as follows:

Response

Status Code: 200 OK

JavaScript

```
{
  "success": true,
  "payload": {
    "id": 15,
    "event": "withdrawal",
    "callback_url":
      "https://webhook-test.com/026b93e74068b30304e45a2488b6f486"
  }
}
```

2.4 Updating the callbackURL of a subscription

To update the callbackURL of a subscription, execute a PATCH request to the /v4/webhooks/{subscriptionId} endpoint.

Request Payload Example:

JavaScript

```
{
  "callback_url":
  "https://webhook.site/aa38ed33-518c-4eb3-b083-3d25af787b066"
}
```

Response

Status Code: 200 OK

JavaScript

```
{
  "success": true,
  "payload": {
    "id": 15,
    "event": "withdrawal",
    "callback_url":
    "https://webhook.site/aa38ed33-518c-4eb3-b083-3d25af787b066"
  }
}
```

2.5 Removing a subscription

To remove a subscription execute a DELETE request to the /v4/webhooks/{subscriptionId} endpoint. A 204 no Content status code should be the response.

Removing a subscription means it will not appear in the list operations and no events will be sent to the callback url registered in it.

Response

Status Code: 204 No content

2.6 Additional information to document

Below is a list of some other behavior not specific to any endpoint in particular, but to how the webhook delivery works at Bitso:

- If a client fails to reply with 200 status codes during event delivery, the system considers the delivery failed and will retry until success or the defined number of retries is reached. The retries may happen with no specific delivery pattern timing
- Order of delivery is not guaranteed: Events can be delivered in an order different from their occurrence
- At least one event is delivered, which means that the same event can be delivered multiple times. Clients should use the event id to filter additional events

2.7 Webhook Events Signature

Webhook signature has a new way to validate the authenticity of the webhook events sent.

We added 2 new headers for each event with the signature:

- **x-bitso-webhook-event-signature:** This is the payload encrypted with our private key for you to decode and validate the signature.
- **x-bitso-key-id:** The identifier for the public key to be used to verify event signature.

Example:

Unset

x-bitso-webhook-event-signature: DMX.....h8Q==

x-bitso-key-id: a1c889ecd24ec289

To validate the sign in the x-bitso-webhook-event-signature, you'll need to use the corresponding public key (You can find this public key at the following endpoint: </v4/webhooks/public-key>) and use the algorithm **SHA256withRSA** to verify the signature of the event.

2.7.1 Public Key Endpoint

This endpoint returns the public key that is used to validate the signature of incoming webhook requests. The public key is necessary to ensure the integrity and authenticity of the data received in the headers.

Response

Status Code: 200 OK

Body:

JavaScript

```
[
  {
    "publicKey": "...",
    "keyId": "abcde"
    "created_at": "2025-02-07T00:00:00+00:00"
  }
]
```

Example Request

Unset

```
curl -X GET "https://bitso.com/v4/webhooks/public-key"
```

3. IP Whitelisting

Our clients should whitelist the IPs we use to send events, those should be easily accessible for clients in our documentation

Environment	IP List
Stage	3.129.233.228 3.22.247.241 3.134.133.168
Production	52.15.91.227 18.216.72.107 18.219.140.132

4. Old subscription renewal

Due to the migration to a new database, we are retrieving all the old subscriptions. The old database will be deprecated soon. If you have not transacted after **February 17, 2025**, your subscription will not be migrated, and you will need to resubscribe using our new **v4** endpoints.

We apologize for any inconvenience this may cause and appreciate your understanding during this migration process. If you have any further questions or concerns, please don't hesitate to reach out to our support team.

4. Webhook retry events

General information

There are retry mechanisms in place for delivering webhook events, in case of multiple failures to deliver, an event may end up not being delivered (usually due to instability in the receiving side). For those cases, clients can make use of this retry mechanism.

Events that were created after the March 20, 2025 date are available for this retry mechanism. Events are available for retrying for a period of 30 days from their creation. The endpoint for retrying events allows specifying an datetime interval for which events matching the criteria will be retried; this interval must not exceed 3 days. Events triggered through this mechanism have a single delivery attempt. Events triggered through this mechanism may not be delivered immediately. The retry of events that were successfully delivered is allowed, this makes it even more important that the best practices about *Handle Duplicate Events* are followed.

Preview amount of events to be retried

Clients can use `/v4/webhooks/events/retry/preview` To identify how many events would be retried for a given criteria without actually triggering the delivery of events. Example:

POST Request to `/v4/webhooks/events/retry/preview`

Request Payload Example:

```
JavaScript
{
  "from_date": "2025-03-18T10:45:20",
  "to_date": "2025-03-18T13:55:20",
  "delivered": true,
  "event": "funding"
```

```
}
```

Where:

- **from_date**: A date (ISO 8601) representing the start of the interval from which events should be fetched
- **to_date**: A date (ISO 8601) representing the end of the interval from which events should be fetched
- **delivered**: Boolean value indicating whether the fetched events were previously delivered successfully
- **event**: The event type to be matched, the option has to be one of the [documented](#).

Response

Status Code: 200 OK

Body:

JavaScript

```
{
  "success": true,
  "payload": {
    "message": "Total items that match the specified criteria: 60"
  }
}
```

Execute retry of events

Clients can use `/v4/webhooks/events/retry/execute` To execute retry of events that match a given criteria. Example:

POST Request to `/v4/webhooks/events/retry/execute`

Request Payload Example:

JavaScript

```
{
  "from_date": "2025-03-18T10:45:20",
  "to_date": "2025-03-18T13:55:20",
  "delivered": true,
  "event": "funding"
}
```

Where:

- `from_date`: A date (ISO 8601) representing the start of the interval from which events should be fetched
- `to_date`: A date (ISO 8601) representing the end of the interval from which events should be fetched
- `delivered`: Boolean value indicating whether the fetched events were previously delivered successfully
- `event`: The event type to be matched, the option has to be one of the [documented](#).

Response

Status Code: 202 Accepted

Body:

JavaScript

```
{
  "success": true,
  "payload": {
    "message": "60 events have been scheduled for retry"
  }
}
```

Errors:

Status Code	Message	Reason
400 - Bad Request	Invalid event type	The event field contains an invalid value, ex: <code>"event": "as"</code>
400 - Bad Request	The <i>fieldName</i> must not be null. Ex: "The from date must not be null"	The fields listed in the error response are missing
400 - Bad Request	From date must be before to date	The interval provided is wrong: the value informed in the to_date can not be after from_date
400 - Bad Request	From date cannot be more than 30 days in the past	We only allow events up to 30 days in the past to be retried
400 - Bad Request	To date cannot be in the future	The interval provided is wrong: the value informed in the to_date field can not be after current date